Co-management LLD, Configuration Blueprint & Troubleshooting Guide

MECM/ConfigMgr + Microsoft Intune + Microsoft Entra ID (Hybrid)

Validation note: Registry values, bit values, error-code mappings and log wording below come from field experience and public documentation. Log snippets are *representative patterns*, not verbatim output; wording varies by ConfigMgr build. Before production use, validate flagged items against Microsoft Learn for your exact ConfigMgr and Windows builds and test in a lab.

---

1. Architecture & Low-Level Design

1.1 Identity Foundation

Microsoft Entra hybrid join (HAADJ) requirements

- Windows 10 (supported builds) or Windows 11, domain-joined, with line of sight to a DC at registration time (or VPN/ DirectAccess; Always On VPN device tunnel recommended).
- **Service Connection Point (SCP)** in AD: `CN=62a0ff2e-97b9-4513-943f-0d221bd30080,CN=Device Registration Configuration,CN=Services,CN=Configuration,DC=...` with `keywords` = `azureADId:<tenantId>` and `azureADName:<verified domain>`. Configure through the Microsoft Entra Connect wizard (*Configure device options*) or `Initialize-ADSyncDomainJoinedComputerSync`.
- **Microsoft Entra Connect Sync** (not Cloud Sync, which does not handle device objects for HAADJ) must sync the computer objects. Devices appear in Entra as *Pending* until the first sync populates the `userCertificate` attribute match and the device completes registration.
- Outbound HTTPS to `enterpriseregistration.windows.net`, `login.microsoftonline.com`, `device.login.microsoftonline.com`, `login.live.com` (device flows), and `graph.microsoft.com`. Bypass TLS inspection for these.
- Registration is driven by the scheduled task `\Microsoft\Windows\Workplace Join\Automatic-Device-Join`. Validate with `dsregcmd /status`: `DomainJoined : YES`, `AzureAdJoined : YES`, `AzureAdPrt : YES`, and a populated `TenantId` / `MdmUrl` after enrollment scope is set.

- Logs: *Applications and Services Logs > Microsoft > Windows > User Device Registration > Admin* (events 204/304/305 etc. indicate discovery or join failures).

Entra Connect synchronization scope

Item	Design requirement
OU/domain filtering	Include every OU that contains computer objects destined for co-management, not only user OUs. Re-run the wizard after OU changes and trigger a delta sync.
Device object filtering	Under *Configure device options* choose *Configure Hybrid Microsoft Entra ID join*. Optionally filter by attribute in the Synchronization Rules Editor (e.g., scope rule on a custom attribute) but document it, as it is a frequent silent cause of *Pending* devices.
Source anchor	`mS-DS-ConsistencyGuid` (default in current versions).
UPN	Sync users with routable UPN suffixes that are verified domains in the tenant. Non-routable suffixes (e.g., `.local`) sync as `onmicrosoft.com` and break SSO and enrollment. Fix by adding a UPN suffix and updating users, or use Alternate Login ID only as a last resort.
Computer name hygiene	Duplicate or stale computer objects create duplicate Entra device records and enrollment ambiguity. Clean up before onboarding.

Seamless SSO (recommended for the user-credential path)

- Enabled in Entra Connect; creates the computer account `AZUREADSSOACC` in each forest. Roll its Kerberos decryption key at least every 30 days (`Update-AzureADSSOForest`).
- Deploy `https://autologon.microsoftazuread-sso.com` to the **Intranet zone** via GPO (Site to Zone Assignment List, value 1).
- Seamless SSO covers browser/legacy flows. HAADJ devices get their Primary Refresh Token (PRT) from the device registration itself. PRT presence is what makes silent, non-interactive enrollment possible.

1.2 Communication Channels

Path	Protocol / Port	Auth / Certificates
ConfigMgr client to MP/SUP/DP (intranet)	HTTPS 443 (recommended, Enhanced HTTP at minimum) or HTTP 80	Site token (Enhanced HTTP) or PKI client auth cert
ConfigMgr client to CMG (internet)	HTTPS 443 to `https://<CMG FQDN>/CCM_Proxy_MutualAuth/<id>`	Entra ID token (device/user on HAADJ/Entra joined), PKI client cert, or site token. Entra ID auth is the common design for co-management
CMG connection point (site server role) to CMG	Outbound HTTPS 443 (verify the port range for your deployment type; the legacy classic cloud service model is retired, so use the VM scale set deployment)	CMG server authentication certificate (public CA wildcard/SAN cert, or Enhanced HTTP-issued model where supported); the connector keeps a persistent outbound tunnel
Site server (SMS Provider/SCP) to Azure	Service connection point in **online** mode over 443	Entra app registrations created at onboarding (Cloud Management/ Cloud Attach)
Device to Intune enrollment	HTTPS 443 to `enterpriseenrollment.manage.microsoft.com`, `enrollment.manage.microsoft.com`	PRT-derived token, device credential

Path	Protocol / Port	Auth / Certificates
Device to Intune MDM service	HTTPS 443 to `*.manage.microsoft.com` (e.g., `manage.microsoft.com`)	MDM client certificate issued at enrollment (`MS-Organization-Access` / MDM device cert)
Push notification	WNS (`*.notify.windows.com`, `client.wns.windows.com`)	Triggers an immediate OMA-DM sync; otherwise default polling (initial rapid cycle, then roughly every 8 hours)
Intune tenant attach / admin-side	Console to Intune/Graph 443	Entra tokens, admin RBAC

Key points: ConfigMgr and Intune are **independent channels**. The CM client talks to ConfigMgr; the MDM agent (`omadmclient` / `DMClient` CSP) talks to Intune. They are coordinated only through (a) the client-side flag set by ConfigMgr policy and (b) the Intune-side recognition of a co-managed device.

1.3 Registry & CSP Mapping

Location	Purpose
`HKLM\SOFTWARE\Microsoft\Enrollments\{GUID}`	One key per MDM enrollment. Check `ProviderID` (should be `MS DM Server`), `EnrollmentState` (1 = enrolled), `EnrollmentType`, `UPN`, `DiscoveryServiceFullURL`, `AADResourceID`.
`HKLM\SOFTWARE\Microsoft\Provisioning\OMADM\Accounts\{GUID}`	OMA-DM account bound to the enrollment; holds the management server address and sync state.
`HKLM\SOFTWARE\Microsoft\Enrollments\{GUID}\FirstSync`	Initial sync status (`IsSyncDone`, `ServerHasFinishedProvisioning`).

Location	Purpose
`HKLM\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\MDM`	Auto-enrollment policy: `AutoEnrollMDM` = 1; `UseAADCredentialType` = 1 (user credential) or 2 (device credential).
`HKLM\SOFTWARE\Microsoft\DeviceManageabilityCSP\Provider\MS_ConfigMgr` (verify exact subkey on your build)	The **DeviceManageability CSP** store where the ConfigMgr agent publishes `ConfigInfo` (agent presence and co-management configuration) so the MDM side can read it.
`HKLM\SOFTWARE\Microsoft\PolicyManager\current\device\*` and `...\providers\{GUID}\default\Device\*`	Policy CSP results from Intune (what is actually applied, per area, plus the provider-specific default). Main place to inspect tattooing.
`HKLM\SOFTWARE\Microsoft\PolicyManager\default\*`	OS default values (what a setting reverts to when MDM policy is removed).
WMI `root\ccm\policy\machine\actualconfig` class `CCM_CoManagementFlags`	The **effective co-management flag set** the ConfigMgr client evaluates locally.
Task Scheduler `\Microsoft\Windows\EnterpriseMgmt\`	Enrollment tasks, including auto-enrollment (`deviceenroller.exe /c /AutoEnrollMDM`) and the per-enrollment GUID sync tasks.

Relevant CSPs: DMClient CSP (enrollment and sync, `./Vendor/MSFT/DMClient/Provider/MS DM Server`), DeviceManageability CSP (`./Vendor/MSFT/DeviceManageability`), Policy CSP (settings), Update CSP and Policy CSP *Update* area (WUfB), Defender CSP (endpoint protection), EnrollmentStatusTracking CSP (ESP), and the Enterprise Modern App Management / Win32 app mechanism through the Intune Management Extension (IME) for apps.

Conflict behavior: where Intune and Group Policy set the same setting, Policy CSP `ControlPolicyConflict/MDMWinsOverGP` governs which wins on supported builds. Co-management does not itself resolve GPO versus Intune conflicts.

1.4 Authority Bitmask Values

The ConfigMgr client receives a single integer, `CoManagementFlags`, in machine policy. Each bit says *"Intune owns this workload"*; unset bits mean ConfigMgr owns it. The client evaluates the integer **locally** after policy download and does not ask Intune.

Bit value	Meaning
1	Co-management / auto-enrollment capability enabled (verify semantics on your build)
2	Compliance policies
4	Resource access policies
8	Device configuration
16	Windows Update policies
64	Client apps
128	Endpoint Protection
256	Office Click-to-Run apps

Example: Compliance + Windows Update + Endpoint Protection to Intune with co-management enabled = 1 + 2 + 16 + 128 = **147**. All current workloads to Intune = 1 + 2 + 4 + 8 + 16 + 64 + 128 + 256 = **479**.

Because the flags can differ per device (pilot vs. production collections), the client's *effective* value is the authoritative one. Read it with:

```
Get-CimInstance -Namespace root\ccm\policy\machine\actualconfig -ClassName
CCM_CoManagementFlags | Format-List *
```

Evaluation sequence on the client: (1) machine policy retrieved from MP/CMG; (2) `CoManagementHandler` evaluates eligibility (MDM enrolled? HAADJ/Entra token valid? tenant configured?); (3) the flag integer is written to `actualconfig`; (4) each handler (`ComplRelayAgent`, `CIAgent`, `WUAHandler`, `EndpointProtectionHandler`, client apps/Office handlers) reads its bit and either executes or skips; (5) `ConfigInfo` is surfaced through the DeviceManageability CSP so the Intune side sees the device as co-managed.

---

2. Step-by-Step Configuration Blueprint

2.1 Prerequisites & Permissions

- **ConfigMgr:** a currently supported Current Branch release (Microsoft supports the latest three; run the newest you can). Co-management requires Windows 10 1709 or later (use supported builds only) or Windows 11 clients with a current CM client.
- **Service Connection Point** in online mode (required for tenant onboarding and update/feature flow).
- **Intune licensing:** Microsoft Intune Plan 1 (standalone, EMS E3/E5, Microsoft 365 E3/E5/F, Business Premium). **Entra ID P1** (included in the suites above) for MDM auto-enrollment and Conditional Access. Users who sign in on the device (user credential path) need the license; with the device credential path, licensing still needs to cover the enrolling identity per Microsoft's current licensing terms, so confirm.
- **CMG (strongly recommended)** for internet-based clients: VM scale set CMG, public-CA server certificate, CMG connection point role, and Entra ID authentication enabled on the site (*Cloud Attach / Azure Services > Azure Active Directory Tenants*).
- **Roles:**
  - Entra ID: **Global Administrator** for the one-time onboarding consent (app registrations). Day-to-day admins need *Intune Administrator* and, for device cleanup, *Cloud Device Administrator*.
  - ConfigMgr: **Full Administrator** security role to run the co-management wizard and change workloads. Delegated admins can later use *Cloud Attach/Co-management* custom roles.
  - Intune: **Intune Administrator** or a custom RBAC role with the right Windows policy and app permissions.
- **Entra MDM scope:** *Entra ID > Mobility (MDM and MAM) > Microsoft Intune*: set **MDM user scope** to *All* or a group containing the pilot users/devices. Default URLs for discovery, terms of use and compliance should remain unchanged. Check *Devices > Enrollment restrictions* (platform, personal device blocking, device limit) so they do not block corporate Windows enrollment.

2.2 Tenant Onboarding

1. In the ConfigMgr console: *Administration > Cloud Services > Co-management* (or *Cloud Attach* for newer versions) and start the **Configure co-management** wizard.
2. Choose the Azure environment (Azure Public, GCC High, etc.) and sign in with a Global Administrator account. This creates the Entra app registrations used by the site for tenant attach and by the CMG.
3. Confirm the **Intune tenant** and that the MDM user scope is set.

4. On *Enablement*: set **Automatic enrollment in Intune** to *Pilot* initially, and select the **Intune Auto Enrollment collection**. Optionally define an exclusion collection (servers, kiosks, special-function devices).
5. On *Workloads*: leave every slider at *Configuration Manager* for the first deployment and set pilot collections per workload only when you are ready to shift.
6. On *Staging*: assign the **pilot collections** for each workload (see 2.4).
7. Finish, then verify under *Monitoring > Co-management* (dashboard shows enrollment status and workload state), and confirm app registrations exist in Entra ID under *App registrations*.
8. Configure the CMG client settings: **Enable clients to use a cloud management gateway** and **Allow access to cloud distribution points** (if used).

2.3 Enrolment Paths

Path 1: Auto-enrolling existing ConfigMgr clients

Prerequisites: devices are HAADJ with a PRT, already have the CM client, and are in the Intune Auto Enrollment collection.

Option A: ConfigMgr-driven. Enabling *Automatic enrollment in Intune* in the co-management properties causes eligible clients to receive a policy that triggers MDM enrollment on their own schedule using device credentials. No GPO is required. Watch `CoManagementHandler.log`.

Option B: Group Policy. Computer Configuration > Administrative Templates > Windows Components > MDM > **Enable automatic MDM enrollment using default Microsoft Entra credentials** (credential type = *User Credential* or *Device Credential*). This writes `AutoEnrollMDM` and `UseAADCredentialType` under `HKLM\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\MDM` and creates the enrollment scheduled task. Use the device credential type for shared or kiosk-like devices, and user credential for standard laptops.

Option C: Manual trigger/ scheduled task. Force it on a test device with:

```
C:\Windows\System32\deviceenroller.exe /c /AutoEnrollMDM
```

Validate: `dsregcmd /status` shows `MdmUrl`, a new key under `HKLM\SOFTWARE\Microsoft\Enrollments`, DeviceManagement-Enterprise-Diagnostics-Provider event 75, and the device in Intune with *Managed by: Co-managed*.

Path 2: Autopilot (modern provisioning)

- **Hybrid Join Autopilot:** deploy the *Intune Connector for Active Directory* (with an account that has rights to create computer objects in the target OU), a *Domain Join* profile, and an Autopilot deployment profile set to *Microsoft Entra hybrid joined*. Devices need line of sight to a DC during OOBE (VPN or wired network). Device enrolls in Intune *first* (MDM-first).
- **Entra-joined Autopilot (cloud-native):** no AD dependency; the ConfigMgr client is added afterward and authenticates through CMG with Entra ID.
- **Installing the ConfigMgr client during/after Autopilot:** package the CM client as a **Win32 app** in Intune and track it in the **Enrollment Status Page (ESP)** if you want it to complete before the user reaches the desktop. For internet-based provisioning, run `ccmsetup.exe` with the CMG management point path and Entra parameters (for example `/mp:https://<CMG FQDN>/CCM_Proxy_MutualAuth/<id>`, `CCMHOSTNAME=<CMG FQDN>/CCM_Proxy_MutualAuth/<id>`, `SMSSiteCode=<site>`, `AADTENANTID=<tenant>`, `AADCLIENTAPPID=<client app id>`, `AADRESOURCEURI=<server app URI>`). Use the exact command line generated by the console (*Cloud Attach/ Co-management* client install guidance) instead of typing it by hand. Alternatively use ConfigMgr's built-in *Intune* deployment of the client in newer releases where available.
- Make Autopilot devices land in a collection that also receives the pilot or production co-management settings, so the CM client arrives with the right workload policy on first check-in.

2.4 Pilot Collections & Phased Rollouts

Collection	Purpose	Notes
`CoMgmt - Intune Auto Enrollment (Pilot)`	Eligible devices that should enroll into Intune	Limit to HAADJ + CM client healthy + OS supported; query-based membership (e.g., Client = 1, Client Active status, OS build).
`CoMgmt - Workload Pilot - <Workload>`	Receives *Intune* ownership for that workload	One per workload so a single workload can be rolled back without affecting others.
`CoMgmt - Production`	Whole fleet once pilot signed off	Switch the workload slider to **Intune** (all) only after pilot sign-off.
`CoMgmt - Exclusions`	Servers, kiosks, labs, critical OT	Excluded from auto-enrollment.

Rollout waves: Wave 0 = IT/lab (10 to 25 devices), Wave 1 = early adopters (1 to 2 percent, mixed hardware and locations), Wave 2 = department rings (10 to 25 percent), Wave 3 = remaining fleet. Hold each ring for at least one full patch cycle and gate promotion on enrollment success rate, compliance state, and help desk ticket trends.

2.5 Workload Shifting Strategy

Recommended order (lowest risk and best value first). Change **one workload at a time** and validate before moving on.

1. **Compliance policies** first. Light risk, and it unlocks Conditional Access based on Intune compliance. Re-create ConfigMgr compliance rules as Intune compliance policies and make sure no CM *Compliance Settings* baselines assigned to the same devices duplicate them. Intune compliance for co-managed devices can include a ConfigMgr **compliance relay** (`ComplRelayAgent`) so the CM client reports CM-side compliance (baselines) into Intune compliance.
2. **Windows Update policies.** Move SUP/WSUS scan behavior to WUfB rings or Windows Autopatch. Prior to the move, remove or neutralize WSUS GPOs and set the scan-source policies deliberately (see Scenario B).
3. **Resource access policies** (VPN, Wi-Fi, certificates, email). Build profiles in Intune and test connectivity before removing the ConfigMgr equivalents.
4. **Endpoint Protection.** Replace ConfigMgr AV, firewall and Defender policies with Intune *Endpoint security* policies (AV, ASR, firewall, EDR onboarding). Make sure Defender is the active AV and no third-party management is also writing settings.
5. **Device configuration.** Move settings to Intune configuration profiles or the Settings Catalog. Inventory every GPO, CI and baseline first, and use Group Policy Analytics in Intune to map settings.
6. **Office Click-to-Run apps.** Route Office updates to Intune (Microsoft 365 Apps deployment and update channel policy).
7. **Client apps** last. Needs the Win32/IME app packaging in place and requires deciding which app catalog is authoritative.

General safeguards: never have the same setting configured by both ConfigMgr and Intune; keep a rollback collection per workload; allow at least 24 to 48 hours (a full policy and sync cycle) for flags and policy to converge; use ConfigMgr *Monitoring > Co-management* and Intune device reports to confirm.

---

3. Troubleshooting Matrix & Log Analysis

3.1 Client-Side Logs

Location for all: `%WinDir%\CCM\Logs`. Open with CMTrace or OneTrace. The patterns below are illustrative.

CoManagementHandler.log (central log for eligibility, enrollment trigger and flag evaluation)

Healthy:

```
Co-management is enabled by policy; workload flags = 147
Device is Hybrid Azure AD joined and has a valid AAD token
MDM enrollment is already present; ProviderID MS DM Server, state 1
CoMgmt capability flags written to actualconfig
```

Failing silent enrollment:

```
Failed to get AAD device token ... 0x...
Enrollment into MDM failed with error 0x8018002b
Device is not eligible: not AAD registered / PRT not available
Waiting for next enrollment retry
```

Look for: policy received (flags value), eligibility checks, the enrollment HRESULT, and retry timing. A missing log or no entries usually means co-management policy never arrived (collection, MP or CMG issue).

ComplRelayAgent.log (compliance relay to Intune)

```
Compliance relay: workload for Compliance is Intune; relaying CM baseline
state
Policy evaluation completed; compliance state posted
Failed to relay compliance state ... error ...
```

Look for: relay posting success, baselines being evaluated, and errors reaching the MDM/DeviceManageability CSP.

CIAgent.log (Compliance Settings: configuration items and baselines)

```
CIAgent: evaluating assigned baselines/CIs
CI evaluation skipped or disabled for workload owned by Intune (verify for
your build)
CI discovery failed: ...
```

Look for: which baselines continue to run after a workload shift, and CIs that still write to the registry or run remediation scripts that conflict with Intune policy.

WUAHandler.log (Windows Update Agent scan source selection)

```
Its a WSUS Update Source type ({GUID}), adding it.
Enabling WUfB / Windows Update scan source
Scan failed with error = 0x80244022
WUA scan completed with 'Windows Update' as the scan source
```

Look for: which source is selected (WSUS vs Windows Update), repeated switching between sources, and WU error codes (0x8024500c, 0x80244022 = connectivity/service; 0x8024402c = proxy/name resolution).

EndpointProtectionHandler.log

```
Endpoint Protection workload owned by Intune; ConfigMgr will not apply EP
policy
EP policy skipped
Antimalware policy applied successfully (workload owned by ConfigMgr)
```

Look for: whether policy is applied or skipped, and Defender provisioning errors. Cross-check with `Get-MpPreference` and `Get-MpComputerStatus`.

3.2 Registry & Event Viewer Checks

- **Enrollment state:** `HKLM\SOFTWARE\Microsoft\Enrollments\{GUID}`: `EnrollmentState` = 1, `ProviderID` = `MS DM Server`. Multiple stale GUIDs (from earlier tenants or failed attempts) can cause trouble. Back up and remove stale keys only after confirming unenrollment.
- **OMA-DM account:** `HKLM\SOFTWARE\Microsoft\Provisioning\OMADM\Accounts\{GUID}`.
- **Auto-enrollment policy:** `HKLM\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\MDM` values `AutoEnrollMDM`, `UseAADCredentialType`.
- **Effective flags:** `CCM_CoManagementFlags` in `root\ccm\policy\machine\actualconfig` (see 1.4).
- **Join state:** `dsregcmd /status` (`AzureAdJoined`, `DomainJoined`, `AzureAdPrt`, `DeviceAuthStatus`, `MdmUrl`). Run it as SYSTEM (`psexec -s`) to see the machine context.
- **Event Viewer:** *Applications and Services Logs > Microsoft > Windows > DeviceManagement-Enterprise-Diagnostics-Provider > Admin*. Event 75 reports auto-enrollment succeeded; event 76 reports it failed, with the Win32 error. Other MDM Enroll events carry HRESULTs from the enrollment server. Also check *User Device Registration > Admin* and *AAD > Operational*.

- **Task scheduler:** `\Microsoft\Windows\EnterpriseMgmt\` tasks should exist and show recent successful runs.
- **Reports:** `mdmdiagnosticstool.exe -area DeviceEnrollment;DeviceProvisioning;Autopilot -zip C:\Temp\mdm.zip` for a full bundle, plus *Settings > Accounts > Access work or school > Info* to generate the Management report.

Frequently seen HRESULTs (confirm against Microsoft's MENROLL error reference for your case):

Code	Typical meaning
`0x8018000a`	Device is already enrolled (often in another tenant/MDM or a stale enrollment).
`0x80180014`	Enrollment blocked by an enrollment restriction (platform/ownership/limit).
`0x80180018`	License or MDM user scope problem for the enrolling identity.
`0x80180026`	Conflicting management agent state; verify the enrollment path and that the device is not blocked by a ConfigMgr/MDM authority conflict.
`0x8018002b`	Generic auto-enrollment failure; most often tied to the user/UPN (non-routable UPN, user not in MDM scope, or no valid token). Read the paired event 76 text and `CoManagementHandler.log`.
`0x8007064a`	Often seen with Entra token or user mismatch problems during auto-enrollment (no MDM scope, mismatched user, or wrong Entra tenant). Treat as a token/identity issue and read the surrounding diagnostics.
`0x801c03f2` / `0x801c0003`	Entra device object missing or user not authorized to join/ device limit reached.

3.3 Common Failure Scenarios & Resolutions

Scenario A: Devices stuck pre-co-managed or failing silent auto-enrollment (0x8018002b, 0x8007064a, etc.)

Symptoms: device shows CM client active but not *Co-managed*; `CoManagementHandler.log` repeats enrollment failures; DeviceManagement event 76.

Diagnostic order:

1. `dsregcmd /status` (SYSTEM context): confirm `AzureAdJoined : YES` and `AzureAdPrt : YES`. If not, fix device registration first (SCP, Entra Connect sync state of the device, outbound access to registration endpoints, time sync).
2. In Entra ID, check the device record: join type *Hybrid Microsoft Entra joined* (not *Pending* and not *Registered*). Remove duplicate records.
3. Confirm the enrolling identity is within **MDM user scope** and has an Intune license (user credential path), and the UPN suffix is routable and verified.
4. Check enrollment restrictions and device limits in Intune.
5. Look for a stale enrollment: multiple `Enrollments\{GUID}` keys, or an enrollment into a different MDM (previous tenant or third-party MDM). Unenroll cleanly and retry (`deviceenroller.exe /c /AutoEnrollMDM`).
6. Confirm the device is in the **Intune Auto Enrollment collection** and has received the co-management policy (check the flag in `actualconfig`).
7. If using GPO: verify `gpresult /h` shows the MDM auto-enrollment policy applying and that the credential type matches the identity available (device credential requires working device token).
8. Check the ConfigMgr side: the Entra tenant is configured, the CMG is healthy, and the client uses Entra authentication (`ClientIdentity` / `LocationServices.log` show the CMG as the management point when off-network).

Scenario B: Policy conflicts and the Dual Scan issue (WSUS vs. Windows Update for Business)

Symptoms: updates show unexpected deferrals, devices scan both WSUS and Windows Update, Feature updates appear that were not approved, or scans flip between sources.

Causes and fixes:

- Legacy dual scan occurs when WUfB deferral policies are present on a device that also has a WSUS server set. Remove WSUS GPOs from co-managed devices once the Windows Update workload moves, and be explicit about the scan source.
- Use the **policy-driven update source** settings (Windows 10 2004 and later and Windows 11): `HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate` values `SetPolicyDrivenUpdateSourceForFeatureUpdates`, `...ForQualityUpdates`, `...ForDriverUpdates`, `...ForOtherUpdates` (0 = Windows Update, 1 = WSUS). The `DisableDualScan` policy is the older control.
- Set the Windows Update workload to Intune for pilot devices only after creating WUfB update rings; then confirm in `WUAHandler.log` that the scan source is Windows Update, and in

`HKLM\SOFTWARE\Microsoft\PolicyManager\current\device\Update` that the Intune values are applied.

- Remove or filter conflicting GPOs (*Specify intranet Microsoft update service location*, deferral and Automatic Updates GPOs). Prefer a clean scope: GPO-free for update behavior on co-managed devices.
- Keep the SUP and the WSUS infrastructure only for devices still on the CM Windows Update workload, and keep third-party update catalogs through ConfigMgr (still scanned through the SUP).
- Validation: `Get-WindowsUpdateLog`, `(New-Object -ComObject Microsoft.Update.ServiceManager).Services | Select Name, IsDefaultAUService`, and the Intune *Update rings* report.

Scenario C: Workload toggle flipped in the console but the client does not respect the switch

Symptoms: workload shows as Intune in the console; device still behaves as ConfigMgr-managed (or vice versa).

Checklist:

1. **Collection scope:** if a pilot collection is configured for that workload, only members get the flag. Verify membership and that the collection updated (full or incremental update).
2. **Policy delivery:** trigger *Machine Policy Retrieval and Evaluation Cycle* and confirm the new flag value is in `CCM_CoManagementFlags`. If it has not changed, check `PolicyAgent.log`, `PolicyEvaluator.log` and MP or CMG connectivity.
3. **Eligibility:** the handler only applies the flag if the device is MDM-enrolled and meets conditions. Confirm in `CoManagementHandler.log`.
4. **Bit math:** decode the integer using the table in 1.4 to confirm the expected bits are set (a common mistake is expecting a workload to move when its collection is not a member of the pilot).
5. **Handler behavior:** check the specific log (`WUAHandler.log`, `EndpointProtectionHandler.log`, `ComplRelayAgent.log`, `CIAgent.log`) for skip or apply messages. A handler can still run if a ConfigMgr deployment is targeting a different (non-workload) object, such as a CI or baseline, or a script.
6. **Intune side:** the Intune policy must exist and be assigned, and the device must have synced (`Settings > Accounts > Access work or school > Info > Sync`). A shifted workload with no Intune policy leaves the device unmanaged for that area.
7. **Stale WMI or corrupt policy:** if flags are right but behavior is wrong, reset policy (`Invoke-WmiMethod -Namespace root\ccm -Class SMS_Client -Name ResetPolicy -ArgumentList 1`) and re-evaluate. Reinstall the CM client only as a last step.

8. Allow time: policy plus sync can take hours. Confirm against the *Monitoring > Co-management* dashboard.

Scenario D: Tattooing when shifting Device Configuration back to ConfigMgr

Symptoms: after moving the workload back, or unassigning an Intune profile, settings from Intune remain on the device.

Why it happens:

- Many Policy CSP settings revert to defaults when the Intune policy is removed, but **ADMX-backed settings, some registry-backed settings, scripts, PowerShell remediation, custom OMA-URI settings and Win32/Settings applied outside Policy CSP** do not always revert and can persist (tattoo).
- ConfigMgr CIs and GPO-registry settings are also persistent unless explicitly reverted.
- Moving the workload only changes **which agent evaluates** policy. It does not remove what was already applied.

Prevention and remediation:

1. Before moving back, record Intune-applied settings (`MDMDiagnosticsTool` report and `HKLM\SOFTWARE\Microsoft\PolicyManager\current\device\*`).
2. Remove or unassign the Intune profile first, allow a sync, then verify the key and value are gone or reverted to the `PolicyManager\default` value.
3. For settings that do not clean up, deploy an explicit revert: a ConfigMgr CI or script that sets the desired value (not just deleting the key), or an Intune remediation (proactive remediation script) that restores the default.
4. For custom OMA-URI, check the CSP's *Delete* behavior; many nodes are not deletable and need a corresponding "Replace" with the neutral value.
5. Avoid GPO and Intune overlap; if overlap exists, set `ControlPolicyConflict/MDMWinsOverGP` deliberately.
6. Where a full reset is needed, a device wipe/ reprovision is the only guaranteed clean state; plan device configuration changes so that a rollback does not depend on tattoo removal.

3.4 Quick Diagnostic Command Set

```
# Join & MDM state (run as SYSTEM)
dsregcmd /status
# Effective co-management flags
Get-CimInstance -Namespace root\ccm\policy\machine\actualconfig -ClassName
CCM_CoManagementFlags | Format-List *
# Enrollment keys
Get-ChildItem HKLM:\SOFTWARE\Microsoft\Enrollments | ForEach-Object { Get-
ItemProperty $_.PSPath } | Select PSChildName, ProviderID, EnrollmentState,
UPN
# Enrollment events
Get-WinEvent -LogName 'Microsoft-Windows-DeviceManagement-Enterprise-
Diagnostics-Provider/Admin' -MaxEvents 50 | Select TimeCreated, Id, Message
# Force trigger
deviceenroller.exe /c /AutoEnrollMDM
# Trigger CM machine policy
Invoke-WmiMethod -Namespace root\ccm -Class SMS_Client -Name TriggerSchedule
-ArgumentList '{00000000-0000-0000-0000-000000000021}'
# MDM diagnostics bundle
mdmdiagnosticstool.exe -area DeviceEnrollment;DeviceProvisioning;Autopilot -
zip C:\Temp\mdm.zip
```

---

4. Operational Recommendations

- Treat every workload shift as a change with a rollback collection and a defined validation checklist.
- Keep ConfigMgr and Intune settings inventories in a single spreadsheet or document to catch overlap before it ships.
- Monitor enrollment success rate, flag convergence time, compliance percentages, and Update ring health weekly during rollout.
- Re-validate prerequisites (versions, licensing, CMG certificate expiry, Entra Connect health, and SSO key rotation) quarterly.

---

5. Under the Hood: What Actually Happens, Phase by Phase

This section walks the full lifecycle of one device, from AD join to a shifted workload, naming the component, the evidence it leaves behind, and the typical failure at each step. Read it alongside the scripts in Section 6. Wording of log lines and some internal names are representative; confirm against your build.

5.1 Phase 0: Hybrid join internals (device registration)

Step	What happens	Evidence	Typical failure
1	The `Automatic-Device-Join` task (`\Microsoft\Windows\Workplace Join`) runs at system start and user sign-in, then retries. It launches `dsregcmd.exe /AutoJoin`.	Task history, last run result `0x0`	Task disabled by hardening scripts or GPO
2	The device reads the **SCP** from AD (LDAP to a DC) to learn the tenant ID and verified domain name.	`dsregcmd /status` > *Diagnostic Data*: `Pre-checks`, `Server error`	No DC reachability, SCP missing in the forest, SCP only in the wrong forest
3	It generates a **self-signed device certificate**, then calls the Device Registration Service (`enterpriseregistration.windows.net`) to create a *pending* device object. The public key is also written to the computer's `userCertificate` attribute in AD.	Entra device record with *Registered = Pending*	Proxy or TLS inspection blocks the endpoint; no outbound path from SYSTEM context
4	**Entra Connect Sync** exports the `userCertificate` value. The service matches it to the pending record and completes the join.	Entra record changes to *Registered* with date; sync cycle shows the computer object exported	Computer OU not in sync scope; Entra Connect device options not configured; filtering rules strip `userCertificate`
5	The device retries, receives its real device certificate, and stores it in `Cert:\LocalMachine\My` (issuer `MS-Organization-Access`).	Certificate exists; `dsregcmd: AzureAdJoined : YES`, `DeviceId` populated	Certificate store ACL or antivirus interference; duplicate cert after re-imaging
6	On user sign-in, **CloudAP** uses the device certificate to request a **Primary Refresh Token (PRT)**.	`dsregcmd: AzureAdPrt : YES`, `AzureAdPrtUpdateTime`	No line of sight at sign-in, time skew, user without sync'd identity

The join can take 30 minutes or more purely because of the Entra Connect sync interval (default 30 minutes). Do not clear *Pending* devices that are younger than one or two sync cycles.

A PRT is **not strictly required** for the ConfigMgr-driven enrollment (which uses a device-based token), but it is essential for user-credential GPO enrollment and for Conditional Access later.

5.2 Phase 1: How the co-management setting reaches the client

1. In the console, co-management properties (auto-enrollment collection, workload pilot collections, tenant app IDs) are stored in the site database and processed by the site components.
2. The Management Point (or CMG-proxied MP) compiles **machine policy** for each client, based on collection membership. The policy includes the `CoManagementFlags` integer and tenant information.
3. On the client, the **Policy Agent** downloads and the **Policy Evaluator** compiles it into WMI. The effective result lands in `root\ccm\policy\machine\actualconfig` (`CCM_CoManagementFlags`). Relevant logs: `PolicyAgent.log`, `PolicyEvaluator.log`, `CcmMessaging.log`, and for internet clients `LocationServices.log` and `ClientLocation.log`.
4. Default machine policy polling is every 60 minutes. A manual *Machine Policy Retrieval & Evaluation Cycle* forces it.

If the flag is wrong at this stage, nothing downstream will behave correctly. Always verify the flag **before** investigating enrollment or workloads.

5.3 Phase 2: `CoManagementHandler` decision logic

When the flag policy changes (and on its own cycle), `CoManagementHandler` evaluates in roughly this order:

1. **Is co-management enabled** by policy? If not, it logs and exits.
2. **OS and client prerequisites:** supported Windows build and a current CM client.
3. **Join state:** device must be Entra hybrid joined (or Entra joined) with a valid device identity. Missing identity results in *not eligible* and a wait.
4. **Tenant match:** the tenant in policy must match the device's tenant.
5. **Enrollment state:** if the device is already MDM-enrolled with `MS DM Server` it skips enrollment; if not, and auto-enrollment is enabled for this client, it proceeds to enroll.
6. **Enrollment call:** the client obtains an Entra token for the MDM resource using device credentials and invokes the Windows MDM registration API (`RegisterDeviceWithManagementUsingAADDeviceCredentials` in `mdmregistration.dll`). Success or HRESULT is written to `CoManagementHandler.log`.
7. **Retry:** on failure it schedules a retry with backoff. Repeated failures are therefore logged on a cadence, not continuously.
8. **Flag publication:** it applies the flag integer to the local handlers and writes `ConfigInfo` for the DeviceManageability CSP.

Because the enrollment API is a **Windows component**, failures return Windows MDM error codes (`MENROLL_E_*`) and also appear in the *DeviceManagement-Enterprise-Diagnostics-Provider/Admin* log, which is usually more informative than the ConfigMgr log.

5.4 Phase 3: The enrollment conversation (MS-MDE2)

Windows modern device enrollment performs three calls to the Intune enrollment service:

1. **Discovery:** the client asks `enrollment.manage.microsoft.com/enrollmentserver/discovery.svc` for service endpoints (policy and enrollment URLs). The address used comes from the `MdmUrl` / `DiscoveryServiceFullURL` configured through the tenant's MDM scope.
2. **Policy (GetPolicies):** the service returns certificate enrollment requirements (key size, algorithm).
3. **Enroll (RequestSecurityToken):** the client submits a certificate request plus an **Entra access token** as proof of identity. The service checks: user or device in **MDM scope**, license, **enrollment restrictions**, device limit, and whether the Entra device object is valid. If everything passes it returns a **provisioning XML** (WAP/CSP payload) containing the issued **MDM device certificate**, the OMA-DM account settings, and initial DMClient settings.

Windows then:

- Installs the MDM certificate in `Cert:\LocalMachine\My` (issuer **Microsoft Intune MDM Device CA**, subject = enrollment device ID).
- Creates `HKLM\SOFTWARE\Microsoft\Enrollments\{GUID}` and `HKLM\SOFTWARE\Microsoft\Provisioning\OMADM\Accounts\{GUID}` .
- Registers the sync tasks under `\Microsoft\Windows\EnterpriseMgmt\{GUID}` .
- Writes `FirstSync` values and starts the first OMA-DM session.

Where failures map: errors before the token is accepted are identity or network problems (Entra, proxy, TLS). Errors after (licensing, scope, restrictions, device limit) are policy problems on the Intune/Entra side and are returned as HRESULTs on the Enroll step. Always capture the **HRESULT plus the exact event text** and the time, then correlate with the Intune *Audit* and *Enrollment failures* report.

5.5 Phase 4: OMA-DM sync and what it carries

- Transport: SyncML over HTTPS to `*.manage.microsoft.com` , authenticated by the MDM device certificate (mutual TLS) established at enrollment.

- Sync cadence (verify against current docs): after enrollment the device polls frequently for a short window (rapid retries over the first ~15 minutes, then every 15 minutes for a period), then settles at about every **8 hours**. A **WNS push** or a manual *Sync* triggers an immediate session.
- Intune sends `Add/Replace/Delete/Get` commands against CSP nodes (Policy CSP, Defender CSP, Update CSP, EnterpriseDesktopAppManagement, DMClient, etc.). The device replies with per-node status codes (200 OK, 404 not found, 418 already exists, 500 error) which Intune shows as *Succeeded / Error / Conflict / Not applicable* in per-setting reports.
- Win32 apps, scripts and remediations travel separately through the **Intune Management Extension (IME)**: IME is installed by a policy sent over OMA-DM (a Win32 app or PowerShell script assignment), then polls the Intune service itself. Logs: `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log` and `AgentExecutor.log`.

5.6 Phase 5: How the co-managed state is shared

- The CM client writes `ConfigInfo` into the DeviceManageability CSP store; Intune reads it during sync. When present, Intune treats the device as co-managed (`managementAgent` = `configurationManagerClientMdm` in Graph) and honors the shifted workloads in its reporting (`configurationManagerClientEnabledFeatures` properties such as `compliancePolicy`, `deviceConfiguration`, `resourceAccess`, `windowsUpdateForBusiness`, `modernApps`).
- ConfigMgr learns of enrollment through client state messages and shows co-management status in *Monitoring > Co-management* (SQL view `v_ClientCoManagementState`, verify exact name/columns in your site database).
- A mismatch between the two (Intune says *MDM only*, ConfigMgr says *not co-managed*, or the reverse) points to a **stale client state message**, a cleared `ConfigInfo`, or a device with two Entra/Intune records.

5.7 Phase 6: What each workload switch does locally

The switch changes **which agent evaluates policy**. It does not push settings and does not remove anything. Each CM handler reads its bit in the flag and either runs or skips.

Workload	CM-side component that stops or continues	Intune-side component that takes over	Key local evidence
Compliance (2)	CM compliance rules do not decide Intune compliance; `ComplRelayAgent` relays CM baseline results into Intune as an additional compliance input	Intune compliance policies evaluated by the MDM agent and the Intune service	`ComplRelayAgent.log`; Intune per-device compliance blade
Resource access (4)	CM VPN, Wi-Fi, certificate and email profiles are not applied	Intune configuration profiles (VPN, Wi-Fi, SCEP/PKCS)	`CertEnrollAgent.log`, `CIAgent.log`, Policy CSP / WiFi CSP, event log *DeviceManagement*
Device configuration (8)	CM stops applying device-configuration policy for this workload; baselines and CIs behave per build (verify)	Intune configuration profiles / Settings Catalog	`CIAgent.log`, `HKLM\...\PolicyManager\current\device`
Windows Update (16)	CM Software Updates deployments are not evaluated for scan and install; the client points scan source at Windows Update	WUfB rings (Policy CSP *Update* area) or Windows Autopatch	`WUAHandler.log`, `UpdatesDeployment.log`, `PolicyManager\current\device\Update`, `WindowsUpdate` policy keys
Client apps (64)	CM continues to deploy CM applications; this workload decides which agent owns app targeting for Intune-managed apps and co-managed device behavior (verify with current docs)	Intune apps (IME for Win32, Enterprise App Management)	IME logs, `AppEnforce.log`, `AppDiscovery.log`

Workload	CM-side component that stops or continues	Intune-side component that takes over	Key local evidence
Endpoint Protection (128)	CM Endpoint Protection agent does not apply anti-malware or firewall policy	Intune *Endpoint security* (Defender CSP, Firewall CSP, ASR)	`EndpointProtectionHandler.log`, `Get-MpPreference`
Office C2R (256)	CM stops managing Microsoft 365 Apps updates through the SUP (the `OfficeMgmtCOM` hand-off is cleared)	Intune Microsoft 365 Apps update policy / Office Cloud Policy	`HKLM\SOFTWARE\Microsoft\Office\ClickToRun\Configuration` (`OfficeMgmtCOM`, `UpdateChannel`), `OfficeC2R` logs

Remember the failure mode this creates: if a workload moves and **no Intune policy exists**, that area simply becomes unmanaged. Likewise if both sides are configured for the same area through non-workload mechanisms (GPO, CI scripts, custom OMA-URI), the last writer wins.

5.8 Phase 7: Windows Update scan source, in detail

1. CM Software Updates normally configures the device's update source as **WSUS** through the local policy interface, pointing at the SUP (`HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate` > `WUServer`, `WUStatusServer`, `UseWUServer`).
2. When WUfB is configured through Intune, Policy CSP writes deferral and update settings under `PolicyManager\current\device\Update` and mirrors them into `HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate`.
3. When both exist, the OS historically scans **both** (dual scan). The control is `DisableDualScan`, or on current Windows builds the **policy-driven scan source** values (`SetPolicyDrivenUpdateSourceForFeatureUpdates`, `...QualityUpdates`, `...DriverUpdates`, `...OtherUpdates`; 0 = Windows Update, 1 = WSUS).
4. With the WU workload set to Intune, the CM client's `WUAHandler` sets the scan source toward Windows Update and stops evaluating CM deployments for the device. If `WUServer` still exists from a **GPO**, the OS may still use it unless the policy-driven source value overrides it.

5.9 Phase 8: Why tattooing happens, at the registry level

- **CSP-backed policy:** the Policy CSP stores desired values in `PolicyManager\providers\{EnrollmentGUID}\default\Device\{Area}` and the effective value in `PolicyManager\current\device\{Area}`. On removal Windows restores the `PolicyManager\default` value, so most plain CSP settings revert cleanly.
- **ADMX-backed policy:** the CSP writes into the traditional policy hive (`HKLM\SOFTWARE\Policies\...`) and records ownership under `PolicyManager\ADMXInstalled`. Removing the MDM policy normally removes the key it owns, but **settings changed by other writers** (GPO, CM CI, script, user) are not reconciled and can persist.
- **Direct writers:** CM CIs, baselines with remediation, PowerShell scripts through IME, and custom OMA-URI nodes that do not have defined delete semantics write once and never clean up.

Therefore tattooing is not a co-management bug. It is the absence of a **revert path** for settings that were applied by a non-reversible mechanism.

5.10 Symptom-to-Root-Cause Matrix

Symptom	Where to look first	Underlying cause	Fix
Device never enrolls; `CoManagementHandler.log` is empty	`PolicyAgent.log`, flag in `actualconfig`	Device not in the enrollment collection, or machine policy not downloaded	Fix collection membership, force policy, check MP/CMG connectivity
Log says *not eligible* / no AAD token	`dsregcmd /status` as SYSTEM	Device is *Pending* or not hybrid joined; no device identity	Fix SCP, Entra Connect sync scope, registration endpoints; wait for sync
Event 76 with `0x8018002b` / `0x8007064a`	DeviceManagement Admin log + `dsregcmd`	Identity or token mismatch (UPN, MDM scope, wrong tenant)	Verify MDM scope, UPN routability, tenant ID, device record uniqueness
`0x8018000a` already enrolled	`HKLM\...\Enrollments`	Existing or stale enrollment; device enrolled elsewhere	Unenroll or clean stale keys (Section 6), retry
`0x80180014` / `0x80180018`	Intune enrollment restrictions and licensing	Policy-side block	Adjust restrictions or license; sync Entra group membership
Enrolled in Intune, shown as *MDM only* (not co-managed)	`ConfigInfo`, CM client health	CM client not reporting `ConfigInfo`, duplicate device record, stale state	Check client health, `CoManagementHandler.log`, remove duplicate Entra/Intune objects
Workload flipped, device ignores it	`CCM_CoManagementFlags`	Device not in pilot collection or old policy	Verify collection, reset policy, decode bit
Updates scan both WSUS and WU	`WUAHandler.log`, WU policy keys	Dual scan: GPO `WUServer` plus WUfB	Remove WSUS GPO, set policy-driven source, validate
Setting remains after profile removal	`PolicyManager` + writer identification	Non-reversible writer (ADMX/CI/script/OMA-URI)	Deploy explicit revert; reprovision if needed

Symptom	Where to look first	Underlying cause	Fix
Intune apps not installing on co-managed device	IME logs, workload state	IME not installed or workload ownership mismatch	Verify IME installation, assignment, workload

---

6. Diagnostic and Remediation Scripts

Usage rules: run client scripts in an elevated PowerShell session (ideally as SYSTEM via `psexec -s -i powershell.exe` or a ConfigMgr script, because `dsregcmd` and the registry show machine context only then). Test every remediation on a lab device first. Scripts that change state use `-WhatIf` / `-Confirm` support. Registry paths, CIM property names and view names are flagged *verify* where they may vary by build.

6.1 Client health check (read-only): `Get-CoMgmtHealth.ps1`

Collects join state, enrollment state, effective workload flags (decoded), enrollment policy, scheduled tasks, certificates, recent MDM events, recent errors from the five key CM logs, and endpoint connectivity. Writes a text report and JSON to `C:\ProgramData\CoMgmtDiag`.

```
#Requires -RunAsAdministrator
[CmdletBinding()]
param(
    [string]$OutDir = "$env:ProgramData\CoMgmtDiag",
    [string]$CmgFqdn,                # optional, e.g. cmg.contoso.com
    [int]$LogTailLines = 400
)
$ErrorActionPreference = 'SilentlyContinue'
New-Item -ItemType Directory -Path $OutDir -Force | Out-Null
$stamp  = Get-Date -Format 'yyyyMMdd-HHmmss'
$txt    = Join-Path $OutDir "CoMgmt-$env:COMPUTERNAME-$stamp.txt"
$json   = Join-Path $OutDir "CoMgmt-$env:COMPUTERNAME-$stamp.json"
$result = [ordered]@{ Computer = $env:COMPUTERNAME; Time = (Get-
Date).ToString('o') }

function Write-Section($t){ "`r`n===== $t =====" | Tee-Object -FilePath $txt
-Append }
function Out-Report($o){ $o | Out-String | Tee-Object -FilePath $txt -Append
| Out-Null }

# --- 1. Join state ----------------------------------------------------------
Write-Section 'Join state (dsregcmd /status)'
$ds = dsregcmd /status
$keys =
```

```
'AzureAdJoined','DomainJoined','EnterpriseJoined','DeviceId','TenantId','Tena
ntName','MdmUrl','AzureAdPrt','AzureAdPrtUpdateTime','DeviceAuthStatus','KeyS
ignTest','NgcSet','WamDefaultSet','TpmProtected'
$dsh = [ordered]@{}
foreach($k in $keys){
  $m = $ds | Select-String -Pattern "^\s*$k\s*:\s*(.+)$" | Select-Object -
First 1
  if($m){ $dsh[$k] = $m.Matches[0].Groups[1].Value.Trim() }
}
Out-Report ([pscustomobject]$dsh)
$result.Join = $dsh
if($dsh.AzureAdJoined -ne 'YES'){ "FINDING: device is not Entra joined
(hybrid join incomplete)." | Tee-Object $txt -Append }
if($dsh.AzureAdPrt -ne 'YES'){ "NOTE: no PRT in this context (expected if run
as SYSTEM; re-check in user context)." | Tee-Object $txt -Append }

# --- 2. Device certificates ------------------------------------------------
Write-Section 'Device certificates (LocalMachine\My)'
$certs = Get-ChildItem Cert:\LocalMachine\My | Where-Object { $_.Issuer -
match 'MS-Organization-Access|Microsoft Intune MDM Device CA|Microsoft Intune
Root' } |
  Select-Object Subject, Issuer, NotBefore, NotAfter, Thumbprint
Out-Report $certs
$result.Certificates = $certs

# --- 3. MDM enrollments ----------------------------------------------------
Write-Section 'MDM enrollments (HKLM\SOFTWARE\Microsoft\Enrollments)'
$enr = Get-ChildItem 'HKLM:\SOFTWARE\Microsoft\Enrollments' | ForEach-Object
{
  $p = Get-ItemProperty $_.PSPath
  if($p.ProviderID){
    [pscustomobject]@{ Key=$_.PSChildName; ProviderID=$p.ProviderID;
State=$p.EnrollmentState; Type=$p.EnrollmentType; UPN=$p.UPN;
Discovery=$p.DiscoveryServiceFullURL; AADResourceID=$p.AADResourceID }
  }
}
Out-Report $enr
$result.Enrollments = $enr
$good = @($enr | Where-Object { $_.ProviderID -eq 'MS DM Server' -and
$_.State -eq 1 })
if($good.Count -eq 0){ 'FINDING: no healthy Intune (MS DM Server) enrollment
found.' | Tee-Object $txt -Append }
if(@($enr | Where-Object ProviderID -eq 'MS DM Server').Count -gt 1){
'FINDING: multiple MS DM Server enrollments (stale entries likely).' | Tee-
Object $txt -Append }
if(@($enr | Where-Object { $_.ProviderID -and $_.ProviderID -ne 'MS DM
```

```
Server' }).Count -gt 0){ 'FINDING: enrollment(s) for another MDM provider
present.' | Tee-Object $txt -Append }

# --- 4. Auto-enrollment policy ------------------------------------------
Write-Section 'Auto-enrollment policy (GPO/MDM keys)'
$mdmPol = Get-ItemProperty
'HKLM:\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\MDM'
Out-Report ($mdmPol | Select-Object AutoEnrollMDM, UseAADCredentialType,
MDMApplicationId)
$result.AutoEnrollPolicy = $mdmPol | Select-Object AutoEnrollMDM,
UseAADCredentialType

# --- 5. Scheduled tasks -------------------------------------------------
Write-Section 'Scheduled tasks (EnterpriseMgmt / Workplace Join)'
$tasks = Get-ScheduledTask -TaskPath
'\Microsoft\Windows\EnterpriseMgmt\*','\Microsoft\Windows\Workplace Join\' |
ForEach-Object {
  $i = $_ | Get-ScheduledTaskInfo
  [pscustomobject]@{ Path=$_.TaskPath; Name=$_.TaskName; State=$_.State;
LastRun=$i.LastRunTime; LastResult=('0x{0:X}' -f $i.LastTaskResult) }
}
Out-Report $tasks
$result.Tasks = $tasks

# --- 6. ConfigMgr client + co-management flags --------------------------
Write-Section 'ConfigMgr client and effective CoManagementFlags'
$svc = Get-Service CcmExec
"CcmExec service: $($svc.Status)" | Tee-Object $txt -Append
$map = [ordered]@{ 1='Capability/AutoEnroll'; 2='Compliance';
4='ResourceAccess'; 8='DeviceConfiguration'; 16='WindowsUpdate';
64='ClientApps'; 128='EndpointProtection'; 256='OfficeC2R' }
$cf = Get-CimInstance -Namespace root\ccm\policy\machine\actualconfig -
ClassName CCM_CoManagementFlags
if($cf){
  Out-Report ($cf | Format-List * | Out-String)
  # property name varies; pick the first numeric property named like *Flag*
  $prop = ($cf | Get-Member -MemberType Property | Where-Object { $_.Name -
match 'Flag' } | Select-Object -First 1).Name
  $val  = [int]$cf.$prop
  "Flags value: $val (property '$prop')" | Tee-Object $txt -Append
  $dec = $map.Keys | ForEach-Object { [pscustomobject]@{ Workload=$map[$_];
Bit=$_; Owner= $(if($val -band $_){'Intune'}else{'ConfigMgr'}) } }
  Out-Report $dec
  $result.Flags = @{ Value=$val; Decoded=$dec }
}else{
  'FINDING: CCM_CoManagementFlags not present. Co-management policy has not
```

```
been received (collection, MP/CMG, or policy issue).' | Tee-Object $txt -
Append
}

# --- 7. Recent MDM events -----------------------------------------------
Write-Section 'DeviceManagement-Enterprise-Diagnostics-Provider/Admin (last
40)'
$ev = Get-WinEvent -LogName 'Microsoft-Windows-DeviceManagement-Enterprise-
Diagnostics-Provider/Admin' -MaxEvents 40 |
  Select-Object TimeCreated, Id, LevelDisplayName, @{n='Message';e=
{($_.Message -replace '\s+',' ').Substring(0,
[Math]::Min(300,$_.Message.Length))}}
Out-Report $ev
$result.MdmEvents = $ev

Write-Section 'User Device Registration/Admin (last 20)'
$ev2 = Get-WinEvent -LogName 'Microsoft-Windows-User Device
Registration/Admin' -MaxEvents 20 |
  Select-Object TimeCreated, Id, LevelDisplayName, @{n='Message';e=
{($_.Message -replace '\s+',' ').Substring(0,
[Math]::Min(300,$_.Message.Length))}}
Out-Report $ev2

# --- 8. CM log scan -----------------------------------------------------
Write-Section 'ConfigMgr log scan (errors / key lines)'
$logDir = Join-Path $env:WinDir 'CCM\Logs'
$logs =
'CoManagementHandler.log','ComplRelayAgent.log','CIAgent.log','WUAHandler.log
','EndpointProtectionHandler.log','PolicyAgent.log','LocationServices.log'
foreach($l in $logs){
  $f = Join-Path $logDir $l
  "--- $l ---" | Tee-Object $txt -Append
  if(Test-Path $f){
    Get-Content $f -Tail $LogTailLines | Select-String -Pattern
'fail|error|0x8[0-9a-f]{7}|not eligible|exception|denied|unable' |
      Select-Object -Last 15 | ForEach-Object { $_.Line.Substring(0,
[Math]::Min(250,$_.Line.Length)) } | Tee-Object $txt -Append
  } else { 'log not found' | Tee-Object $txt -Append }
}

# --- 9. Connectivity ----------------------------------------------------
Write-Section 'Connectivity (TCP 443)'
$targets =
'enterpriseregistration.windows.net','login.microsoftonline.com','device.logi
n.microsoftonline.com','enterpriseenrollment.manage.microsoft.com','enrollmen
t.manage.microsoft.com','manage.microsoft.com','graph.microsoft.com','client.
```

```
wns.windows.com'
if($CmgFqdn){ $targets += $CmgFqdn }
$conn = foreach($t in $targets){
  $r = Test-NetConnection -ComputerName $t -Port 443 -WarningAction
SilentlyContinue
  [pscustomobject]@{ Host=$t; Resolved=($r.RemoteAddress -as [string]);
TcpOk=$r.TcpTestSucceeded }
}
Out-Report $conn
$result.Connectivity = $conn
$conn | Where-Object { -not $_.TcpOk } | ForEach-Object { "FINDING: cannot
reach $($_.Host):443 from this device." | Tee-Object $txt -Append }

# --- 10. Time sync ----------------------------------------------------------
Write-Section 'Time'
w32tm /query /status | Tee-Object $txt -Append | Out-Null

$result | ConvertTo-Json -Depth 6 | Set-Content $json
"`r`nReport: $txt`r`nJSON  : $json"
```

How to read it: work top to bottom. Join state first (no hybrid join, nothing else matters). Then enrollment (a healthy `MS DM Server` entry with state 1). Then flags (policy arrived and decoded as you expect). Then events and logs (the *why*). Connectivity last, to explain network-class errors.

6.2 Targeted decoders and log filters

Decode a flag value (any machine, no dependencies):

```
function Convert-CoMgmtFlags {
    param([Parameter(Mandatory)][int]$Flags)
    $map = [ordered]@{ 1='Capability/AutoEnroll'; 2='Compliance';
4='ResourceAccess'; 8='DeviceConfiguration'; 16='WindowsUpdate';
64='ClientApps'; 128='EndpointProtection'; 256='OfficeC2R' }
    $map.Keys | ForEach-Object {
        [pscustomobject]@{ Workload=$map[$_]; Bit=$_; Owner = $(if($Flags -
band $_){'Intune'}else{'ConfigMgr'}) }
    }
}
Convert-CoMgmtFlags -Flags 147 | Format-Table -AutoSize
```

Trace the enrollment timeline from `CoManagementHandler.log`:

```
$log = "$env:WinDir\CCM\Logs\CoManagementHandler.log"
Get-Content $log -Tail 2000 |
  Select-String 'flag|enrol|token|eligible|MDM|tenant|0x8' |
  ForEach-Object { $_.Line } | Select-Object -Last 60
```

Pull only the relevant MDM events with decoded HRESULT hints:

```
$hints = @{
 '0x8018000a'='Already enrolled (stale or other-MDM enrollment)'
 '0x80180014'='Blocked by enrollment restriction'
 '0x80180018'='License or MDM user scope problem'
 '0x80180026'='Conflicting management state (verify)'
 '0x8018002b'='Auto-enrollment failure; check UPN, MDM scope, token'
 '0x8007064a'='Identity/token mismatch during auto-enrollment (verify)'
 '0x801c03f2'='Entra device object missing'
 '0x801c0003'='User not authorized to join or device limit' }
Get-WinEvent -LogName 'Microsoft-Windows-DeviceManagement-Enterprise-
Diagnostics-Provider/Admin' -MaxEvents 200 |
 Where-Object { $_.Id -in 75,76,72,208,209 -or $_.LevelDisplayName -in
'Error','Warning' } |
 ForEach-Object {
   $hit = ($hints.Keys | Where-Object { $_.Message -match $_ })
   $code = ([regex]::Match($_.Message,'0x[0-9a-fA-F]{8}')).Value.ToLower()
   [pscustomobject]@{ Time=$_.TimeCreated; Id=$_.Id; Code=$code;
Hint=$hints[$code]; Msg=($_.Message -replace '\s+',' ').Substring(0,
[Math]::Min(180,$_.Message.Length)) }
 } | Format-Table -AutoSize -Wrap
```

6.3 Remediation: repair a failed or stuck MDM enrollment: `Repair-MdmEnrollment.ps1`

Safe, staged repair. Stage 1 refreshes policy and re-triggers enrollment. Stage 2 (opt-in) backs up and removes **stale** enrollment entries (those not in state 1 and not backed by a working account). It never touches a healthy enrollment.

```
#Requires -RunAsAdministrator
[CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')]
param(
    [switch]$RemoveStaleEnrollments,
    [string]$BackupDir = "$env:ProgramData\CoMgmtDiag\backup",
    [int]$WaitSeconds = 120
)
$ErrorActionPreference = 'Stop'
New-Item -ItemType Directory -Path $BackupDir -Force | Out-Null
$stamp = Get-Date -Format 'yyyyMMdd-HHmmss'
```

```
# Stage 0: preconditions
$ds = dsregcmd /status
if(($ds | Select-String 'AzureAdJoined\s*:\s*YES') -eq $null){
  throw 'Device is not Entra joined. Fix hybrid join first; enrollment cannot
succeed.'
}

# Stage 1: refresh policy and re-trigger
if($PSCmdlet.ShouldProcess($env:COMPUTERNAME,'Run gpupdate and trigger MDM
auto-enrollment')){
  gpupdate /force | Out-Null
  # Re-create enrollment task if missing by triggering the enroller
  Start-Process -FilePath "$env:WinDir\System32\deviceenroller.exe" -
ArgumentList '/c','/AutoEnrollMDM' -Wait -NoNewWindow
}

# Optional Stage 2: clean stale entries
if($RemoveStaleEnrollments){
  $root = 'HKLM:\SOFTWARE\Microsoft\Enrollments'
  $stale = Get-ChildItem $root | ForEach-Object {
    $p = Get-ItemProperty $_.PSPath
    if($p.ProviderID -eq 'MS DM Server' -and $p.EnrollmentState -ne 1){ $_ }
  }
  foreach($k in $stale){
    $guid = $k.PSChildName
    $bak  = Join-Path $BackupDir "Enrollments-$guid-$stamp.reg"
    if($PSCmdlet.ShouldProcess($guid,'Back up and remove stale enrollment
key')){
      reg export "HKLM\SOFTWARE\Microsoft\Enrollments\$guid" $bak /y | Out-
Null
      Remove-Item $k.PSPath -Recurse -Force
      # remove matching scheduled tasks if present
      Get-ScheduledTask -TaskPath "\Microsoft\Windows\EnterpriseMgmt\$guid\*"
-ErrorAction SilentlyContinue |
        Unregister-ScheduledTask -Confirm:$false
      Write-Host "Removed stale enrollment $guid (backup: $bak)"
    }
  }
  # Re-trigger after cleanup
  Start-Process -FilePath "$env:WinDir\System32\deviceenroller.exe" -
ArgumentList '/c','/AutoEnrollMDM' -Wait -NoNewWindow
}

# Stage 3: wait and verify
$deadline = (Get-Date).AddSeconds($WaitSeconds)
```

```
do {
  Start-Sleep 10
  $ok = Get-ChildItem 'HKLM:\SOFTWARE\Microsoft\Enrollments' | ForEach-Object
{ Get-ItemProperty $_.PSPath } |
        Where-Object { $_.ProviderID -eq 'MS DM Server' -and
$_.EnrollmentState -eq 1 }
} until($ok -or (Get-Date) -gt $deadline)
if($ok){ Write-Host 'SUCCESS: healthy Intune enrollment present.' -
ForegroundColor Green }
else   { Write-Warning 'Not enrolled yet. Review DeviceManagement event log
(events 75/76) and CoManagementHandler.log.' }
```

Run `Repair-MdmEnrollment.ps1 -WhatIf` first. Use `-RemoveStaleEnrollments` only when the diagnostic report shows multiple or non-healthy `MS DM Server` entries. If the device is enrolled in a *different* MDM or tenant, unenroll it properly (Settings > Accounts > Access work or school, or the previous service) instead of deleting keys.

6.4 Remediation: force ConfigMgr policy and co-management re-evaluation

```
# 1. Machine policy retrieval + evaluation
'{00000000-0000-0000-0000-000000000021}','{00000000-0000-0000-0000-
000000000022}' | ForEach-Object {
  Invoke-CimMethod -Namespace root\ccm -ClassName SMS_Client -MethodName
TriggerSchedule -Arguments @{ sScheduleID = $_ }
}
Start-Sleep 60
# 2. Re-check flag
Get-CimInstance -Namespace root\ccm\policy\machine\actualconfig -ClassName
CCM_CoManagementFlags | Format-List *

# 3. If the flag is stale or corrupt, reset policy (heavier; re-downloads all
policy)
# Invoke-CimMethod -Namespace root\ccm -ClassName SMS_Client -MethodName
ResetPolicy -Arguments @{ uFlags = [uint32]1 }
# Restart-Service CcmExec
```

If the flag still does not match the console after a reset, the cause is on the server side (collection membership, site policy, MP/CMG). Check collection membership, `CMGService` / `CMGSetup` logs for CMG clients, and MP logs (`MP_Policy.log` , `MP_Location.log`).

6.5 Remediation: detect and fix Windows Update dual-scan: `Test-WuScanSource.ps1`

```
#Requires -RunAsAdministrator
$wu  = 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate'
$au  = "$wu\AU"
$pm  = 'HKLM:\SOFTWARE\Microsoft\PolicyManager\current\device\Update'
$p   = Get-ItemProperty $wu -ErrorAction SilentlyContinue
$a   = Get-ItemProperty $au -ErrorAction SilentlyContinue
$m   = Get-ItemProperty $pm -ErrorAction SilentlyContinue

[pscustomobject]@{
  WUServer                     = $p.WUServer
  UseWUServer                  = $a.UseWUServer
  DisableDualScan              = $p.DisableDualScan
  SourceForQuality (0=WU,1=WSUS) =
$p.SetPolicyDrivenUpdateSourceForQualityUpdates
  SourceForFeature              =
$p.SetPolicyDrivenUpdateSourceForFeatureUpdates
  SourceForDriver               =
$p.SetPolicyDrivenUpdateSourceForDriverUpdates
  SourceForOther                =
$p.SetPolicyDrivenUpdateSourceForOtherUpdates
  DeferQualityDays             = $m.DeferQualityUpdatesPeriodInDays
  DeferFeatureDays             = $m.DeferFeatureUpdatesPeriodInDays
  BranchReadiness              = $m.BranchReadinessLevel
} | Format-List

# Which service does the OS treat as default?
$sm = New-Object -ComObject Microsoft.Update.ServiceManager
$sm.Services | Select-Object Name, IsDefaultAUService, OffersWindowsUpdates,
ServiceID | Format-Table -AutoSize

# What did the CM client decide?
Get-Content "$env:WinDir\CCM\Logs\WUAHandler.log" -Tail 300 | Select-String
'WSUS|WUfB|Update Source|scan|0x8'

# Findings
if($p.WUServer -and $m){ Write-Warning 'WSUS server is set while Intune/WUfB
policy is present: dual scan risk.' }
if($null -eq $p.SetPolicyDrivenUpdateSourceForQualityUpdates -and
$p.WUServer){ Write-Warning 'No policy-driven source set; the OS may still
use WSUS or both.' }
```

Remediation guidance:

1. Remove the WSUS GPO (*Specify intranet Microsoft update service location*) and deferral/automatic-updates GPOs for the co-managed OU, rather than overriding them locally.
2. Create an Intune *Settings catalog* profile that sets the four policy-driven source values to **Windows Update**, assigned to the devices whose WU workload is Intune.
3. Keep devices with the WU workload on ConfigMgr on a different GPO scope so they do not receive Intune WUfB settings.
4. After the next policy cycle, rerun the script and confirm the Windows Update service is the active source and `WUAHandler.log` no longer toggles back.

6.6 Tattoo audit and controlled revert: `Get-PolicyTattoo.ps1`

Inventories what MDM currently applies and where it came from, so you can decide what must be reverted before moving Device Configuration back to ConfigMgr.

```
#Requires -RunAsAdministrator
[CmdletBinding()]
param([string]$OutCsv =
"$env:ProgramData\CoMgmtDiag\PolicyAudit-$env:COMPUTERNAME.csv")

$enrollGuids = Get-ChildItem 'HKLM:\SOFTWARE\Microsoft\Enrollments' | Where-
Object {
  (Get-ItemProperty $_.PSPath).ProviderID -eq 'MS DM Server' } | ForEach-
Object PSChildName

$rows = New-Object System.Collections.Generic.List[object]

# 1. Currently effective MDM policy values
$cur = 'HKLM:\SOFTWARE\Microsoft\PolicyManager\current\device'
Get-ChildItem $cur -ErrorAction SilentlyContinue | ForEach-Object {
  $area = $_.PSChildName
  $props = Get-ItemProperty $_.PSPath
  $props.PSObject.Properties | Where-Object { $_.Name -notmatch '^PS' -and
$_.Name -notmatch '_(ProviderSet|WinningProvider|LastWrite)$' } | ForEach-
Object {
    $n = $_.Name
    $winning = $props."${n}_WinningProvider"
    $rows.Add([pscustomobject]@{ Source='PolicyManager.current'; Area=$area;
Setting=$n; Value=$_.Value; WinningProvider=$winning })
  }
}

# 2. Provider-level values per enrollment
foreach($g in $enrollGuids){
```

```
  $prov =
"HKLM:\SOFTWARE\Microsoft\PolicyManager\providers\$g\default\Device"
  Get-ChildItem $prov -ErrorAction SilentlyContinue | ForEach-Object {
    $area = $_.PSChildName
    (Get-ItemProperty $_.PSPath).PSObject.Properties | Where-Object { $_.Name
-notmatch '^PS' } | ForEach-Object {
      $rows.Add([pscustomobject]@{ Source="Provider:$g"; Area=$area;
Setting=$_.Name; Value=$_.Value; WinningProvider=$g })
    }
  }
}

# 3. ADMX-backed policy hive (potential tattoo source)
$admx = 'HKLM:\SOFTWARE\Microsoft\PolicyManager\ADMXInstalled'
if(Test-Path $admx){ Get-ChildItem $admx -Recurse -ErrorAction
SilentlyContinue | ForEach-Object {
  $rows.Add([pscustomobject]@{ Source='ADMXInstalled'; Area=$_.PSChildName;
Setting=$_.Name; Value=''; WinningProvider='' }) } }

$rows | Export-Csv $OutCsv -NoTypeInformation
"Exported $($rows.Count) rows to $OutCsv"
$rows | Group-Object Area | Sort-Object Count -Descending | Select-Object -
First 15 Name, Count | Format-Table -AutoSize
```

Controlled revert procedure (before moving Device Configuration back):

1. Run the audit on a representative device and on a device that has *never* had Intune profiles. Diff the CSVs: the extra rows are what Intune applied.
2. In Intune, unassign (do not delete yet) the profiles. Sync and re-run the audit. Rows that vanish were CSP-reversible.
3. For the rows that remain, identify the writer: ADMX-backed (check `HKLM\SOFTWARE\Policies\...`), a script, a CI, or custom OMA-URI.
4. Deploy explicit reversions through the owner you intend to keep: a CM configuration item with a *remediate* action, or an Intune remediation script. Revert to the **intended value**, not just delete the key, and keep a rollback item for 2 to 4 weeks.
5. Only then move the Device Configuration workload to ConfigMgr and delete the Intune profiles.

Example remediation (detection plus fix) for one registry-backed tattoo:

```
# Detection
$p = 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\Example'; $n =
'ExampleSetting'
$v = (Get-ItemProperty $p -ErrorAction SilentlyContinue).$n
if($null -ne $v){ exit 1 } else { exit 0 }   # non-zero = remediation needed

# Remediation
Remove-ItemProperty -Path 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\Example'
-Name 'ExampleSetting' -ErrorAction SilentlyContinue
```

6.7 Cloud-side view: Entra and Intune via Microsoft Graph

Finds pending hybrid devices, duplicate device records and management-agent mismatches in bulk. Requires the Microsoft Graph PowerShell SDK and read permissions.

```
Connect-MgGraph -Scopes
'Device.Read.All','DeviceManagementManagedDevices.Read.All'

# Entra hybrid-joined device records
$aad = Get-MgDevice -All -ConsistencyLevel eventual -Filter "trustType eq
'ServerAd'" `
   -Property
Id,DeviceId,DisplayName,TrustType,RegistrationDateTime,ApproximateLastSignInD
ateTime,AccountEnabled,OperatingSystem,OperatingSystemVersion

# 1. Pending (no registration time) - verify definition in your tenant
$pending = $aad | Where-Object { -not $_.RegistrationDateTime }
"Pending hybrid devices: $($pending.Count)"

# 2. Duplicates by display name
$dups = $aad | Group-Object DisplayName | Where-Object Count -gt 1
"Duplicate names: $($dups.Count)"
$dups | ForEach-Object { $_.Group | Select-Object DisplayName, DeviceId,
RegistrationDateTime, ApproximateLastSignInDateTime }

# 3. Intune management agents
$mdm = Get-MgDeviceManagementManagedDevice -All -Filter "operatingSystem eq
'Windows'" `
   -Property
DeviceName,ManagementAgent,AzureAdDeviceId,ComplianceState,LastSyncDateTime
$mdm | Group-Object ManagementAgent | Select-Object Name, Count   # look for
configurationManagerClientMdm

# 4. Entra devices with no Intune record (candidates for failed enrollment)
$mdmIds = $mdm.AzureAdDeviceId
```

```
$aad | Where-Object { $_.DeviceId -notin $mdmIds -and $_.AccountEnabled } |
  Select-Object DisplayName, DeviceId, ApproximateLastSignInDateTime |
  Sort-Object ApproximateLastSignInDateTime -Descending | Select-Object -
First 50

# 5. Stale Intune records (no sync > 30 days)
$mdm | Where-Object { $_.LastSyncDateTime -lt (Get-Date).AddDays(-30) } |
  Select-Object DeviceName, ManagementAgent, LastSyncDateTime | Sort-Object
LastSyncDateTime
```

6.8 Server-side: ConfigMgr site database queries

Run read-only against the site database. View and column names can differ by version, so start with `SELECT TOP 5 *` to confirm.

```
-- Co-management state per client (verify view/columns)
SELECT TOP 5 * FROM v_ClientCoManagementState;

-- Clients that have the CM client but no Entra identity known to ConfigMgr
SELECT sys.Name0, sys.Client0, sys.Client_Type0, sys.AAD_Device_ID0
FROM v_R_System sys
WHERE sys.Client0 = 1 AND (sys.AAD_Device_ID0 IS NULL OR sys.AAD_Device_ID0 =
'');

-- Collection members of the enrollment collection with last policy request
time
SELECT sys.Name0, cs.LastPolicyRequest, cs.LastDDR, cs.LastActiveTime
FROM v_FullCollectionMembership fcm
JOIN v_R_System sys ON sys.ResourceID = fcm.ResourceID
LEFT JOIN v_CH_ClientSummary cs ON cs.ResourceID = fcm.ResourceID
WHERE fcm.CollectionID = 'XXX00123'   -- your auto-enrollment collection
ORDER BY cs.LastPolicyRequest;
```

If `AAD_Device_ID0` is empty for devices that are hybrid joined in Entra, the CM client is not reporting its Entra identity; check client version, `ClientIDManagerStartup.log` and `CcmMessaging.log`.

6.9 Which script to run for which symptom

Symptom	Run	Next step if it still fails
Device not co-managed	`Get-CoMgmtHealth.ps1`	Follow the first FINDING in the report; then Section 5.10

Symptom	Run	Next step if it still fails
Event 76 / `0x8018002b`	6.2 (HRESULT filter) then `Repair-MdmEnrollment.ps1 -WhatIf`	6.7 (Entra device record and duplicates), MDM scope and licensing
`0x8018000a`	`Get-CoMgmtHealth.ps1` (stale entries) then Repair with `-RemoveStaleEnrollments`	Verify no other MDM enrollment is intended
Workload ignored	6.4 then re-run health check	Server-side collection and policy (6.8)
Duplicate update scan	6.5	Remove WSUS GPO, deploy policy-driven source profile
Settings persisting	6.6	Explicit revert via CI or remediation
Fleet-level view	6.7 and 6.8	Group findings by cause and fix in bulk

6.10 Evidence checklist for a Microsoft support case

Collect: output of `Get-CoMgmtHealth.ps1`; `mdmdiagnosticstool.exe -area DeviceEnrollment;DeviceProvisioning;Autopilot -zip C:\Temp\mdm.zip`; the CM logs listed in Section 3.1 (zipped, with timestamps of the repro); `dsregcmd /status` output; Entra device ID, Intune device ID, enrollment correlation ID from event 76 or the Intune enrollment failure report; and the exact UTC time of the last attempt.